

5 ejercicios para la clase de repaso

Los enunciados están **reconstruidos** a partir de los datos y comentarios de cada script (en el repositorio está el código, no el enunciado original). Conviene contrastar la redacción con el enunciado que dieron los profesores.

1 Sistema lineal por Jacobi — “Proveedores de helado”

`1f_python/heladojacobi.py` ↗

Método: Jacobi (Rutina 002)

ENUNCIADO

Una heladería se abastece de **4 proveedores (A, B, C, D)**. Cada proveedor entrega un producto compuesto por 4 ingredientes en distintas proporciones (la matriz de coeficientes, en %). Para cubrir la producción se necesitan las cantidades objetivo del vector **B**. Determinar **cuántos kg pedir a cada proveedor** resolviendo el sistema $A \cdot x = B$ por el método de **Jacobi** con `tol = 1e-2`.

Cómo leer la matriz: fila vs columna

	Prov. A	Prov. B	Prov. C	Prov. D
Ecuación 1 →	0.60	0.08	0.19	0.11
Ecuación 2 →	0.14	0.65	0.15	0.12
Ecuación 3 →	0.11	0.17	0.60	0.12
Ecuación 4 →	0.15	0.10	0.06	0.65

- **Cada fila** (horizontal) es **una ecuación completa** del sistema: son los coeficientes que, sumados, deben dar el valor de **B** de esa fila.
- ↓ **Cada columna** (vertical) agrupa los coeficientes que multiplican a **una misma incógnita** — acá, un mismo proveedor. La columna 1 “pertenece” a $x[0]$, la 2 a $x[1]$, etc.
- El elemento $A[i, j]$ es el cruce de la **fila i** (ecuación) con la **columna j** (incógnita). La **diagonal** $A[i, i]$ es el coeficiente de la incógnita propia de esa ecuación — la que Jacobi despeja.
- El vector **B** y la solución **x** van en **1D** (4,) — una sola fila de números —, **nunca** como columna (4,1), o la rutina de Jacobi se rompe.

Tip — Si A no es diagonalmente dominante (lo más común)

Para que Jacobi (y Gauss-Seidel) converjan conviene que **A** sea **diagonalmente dominante**: en cada fila, el |elemento de la diagonal| debe superar a la suma de los |demás|. Si la A que te dan no lo cumple, casi siempre se puede **reordenar** para lograrlo. Regla práctica: hacé intercambios **puros** — o solo filas, o solo columnas, **nunca mezclados** (mezclar los dos es un lío para rastrear y te equivocás seguro).

- **Si intercambias FILAS** (reordenás ecuaciones): tenés que intercambiar **también las componentes de B** en el mismo orden — B viaja pegado a las filas. La solución **x** **no cambia de orden**: $x[0]$ sigue siendo la primera incógnita.
- **Si intercambias COLUMNAS** (reordenás incógnitas): B **no se toca** (B va por ecuación = fila, no por incógnita = columna). Pero la solución sale **permutada**: al terminar tenés que **volver a ordenar** sus componentes para saber cuál corresponde a cada incógnita original.
- **En este ejercicio puntual** la A ya es diagonalmente dominante, así que no hace falta reordenar; el tip vale para los muchos ejercicios donde no lo es.

DATOS

Proporciones (matriz A, ya dividida $\div 100$):

```
[[0.60, 0.08, 0.19, 0.11],  
 [0.14, 0.65, 0.15, 0.12],  
 [0.11, 0.17, 0.60, 0.12],  
 [0.15, 0.10, 0.06, 0.65]]
```

Requerimientos B = [34, 52, 56, 59]

Tolerancia tol = 1e-2

Valor inicial x = [0, 0, 0, 0]

SE PIDE

Los kg a pedir a cada proveedor (A, B, C, D) y el error absoluto final.

2 Interpolación de Lagrange — “Parámetros PID de impresión 3D”

[1f_python/filamentos.py](#)

Lagrange (Rutina 012) + Falsa posición (Rutina 005)

ENUNCIADO

Los parámetros de control PID (**Kp**, **Ki**, **Kd**) de una impresora 3D dependen de la **temperatura del filamento**. Se dispone de 8 mediciones tabuladas de (T, Kp, Ki, Kd).

a) Hallar los parámetros PID para imprimir **PLA a 170 °C** (temperatura que **no** está tabulada), construyendo un polinomio interpolador de **Lagrange** para cada parámetro y evaluándolo en 170.

b) Hallar el **valor máximo de Ki** y la temperatura a la que ocurre (derivada del polinomio igualada a 0, resuelta por **falsa posición** en [200, 210]).

DATOS

```
T = [105, 108, 157, 168, 183, 218, 221, 236]
Kp = [37.24, 51.19, 66.07, 79.33, 34.28, 60.96, 87.91, 13.31]
Ki = [2.356, 1.667, 1.25, 1.07, 3.04, 3.981, 0.763, 3.129]
Kd = [51.96, 96.57, 85.93, 64.76, 81.82, 68.91, 86.3, 60.96]
```

```
T_PLA = 170      tol = 1e-8
```

SE PIDE

Kp, Ki y Kd interpolados en 170 °C; el valor máximo de Ki y la temperatura a la que se alcanza.

3 Mínimos cuadrados — “Crecimiento de bacterias”

[1f_python/bacteriasminimos.py](#)

Newton multidimensional (Rutina 009) + R² (Rutina 010)

ENUNCIADO

La tasa de crecimiento **k** de una población de bacterias en función de la concentración de sustrato **c** sigue un modelo de saturación:

$$k(c) = (k_{\max} \cdot c^2) / (c_s + c^2)$$

Se tienen 5 mediciones experimentales (c, k) .

- Ajustar los parámetros **kmax** y **cs** por **mínimos cuadrados** (minimizando la suma de cuadrados del error con Newton multidimensional aplicado sobre el gradiente).
- Evaluar la calidad del ajuste con el **coeficiente R²**.
- Estimar **k para c = 2**.

DATOS

```
c = [0.5, 0.8, 1.5, 2.5, 4]
k = [0.7, 1.6, 3.9, 6.1, 7.6]
```

```
Punto inicial  P0 = [kmax, cs] = [8, 3]
Tolerancia      tol = 1e-12
```

SE PIDE

Los parámetros kmax y cs, el coeficiente R², y la estimación de k en c = 2.

4 Integración por Simpson — “Hallar K para un área dada”

[2p_python/AV6_2.py](#)

Simpson 1/3 (Rutina 015)

ENUNCIADO

Hallar el valor de la constante **K** tal que el **área bajo la curva**

$$f(x) = 5 \cdot x^2 + (2K + 10) \cdot x + (2K + 31)$$

entre $x = 7$ y $x = 9$ sea igual a **100**. Aproximar el área con **Simpson 1/3** ($n = 10$). Reportar además el **error real** en el área (respecto de 100) y el **error máximo** debido a la tolerancia.

Idea clave (buena para mostrar): la integral es **lineal**, así que el área se parte en $\text{Área} = \text{Área_conocida} + K \cdot \text{Área_del_término_en_K}$, y de ahí se **despeja K** directamente sin resolver nada iterativo.

DATOS

$$f(x) = 5x^2 + (2K+10)x + (2K+31)$$

$[a, b] = [7, 9]$ Área objetivo = 100
 $n = 10$ tol = 1e-2

SE PIDE

El valor de K, el área verdadera con esa K, el error real y el error máximo por tolerancia.

5

Optimización con multiplicadores de Lagrange — “Silo de costo mínimo”

[1f_python/silolagrange.py](#)

Multiplicadores de Lagrange + Newton multidimensional (Rutina 009)

ENUNCIADO

Se quiere diseñar un **silo** formado por un **cilindro** rematado por una **semiesfera**, apoyado sobre una **base circular**. Debe contener un volumen de **1000 m³**. Los costos por m² de cada superficie son: cilindro **10**, semiesfera **25**, base **30** (usd/m²). Hallar el **radio** y la **altura** que **minimizan el costo total**, usando **multiplicadores de Lagrange** ($\nabla L = 0$, con la restricción de volumen) y resolviendo el sistema resultante por **Newton multidimensional**.

DATOS

Costos: cilindro = 10, semiesfera = 25, base = 30 (usd/m²)
 Volumen requerido = 1000 m³
 $\text{Volumen} = \pi \cdot r^2 \cdot h + \frac{1}{2} \cdot (4/3) \cdot \pi \cdot r^3$ (cilindro + semiesfera)
 Punto inicial $P_0 = [r, h, \lambda] = [4, 15, -5]$
 Tolerancia tol = 1e-1

SE PIDE

El radio, la altura y el costo mínimo.